Lesson 10: Play It Again

Godot Engine

VIDEO GAME CREATION

This week: when the round ends, the game actually stops - and pressing R brings it back to a genuinely fresh start.

Open your own [YourName]Game project from Lesson 9 - don't start a new one.

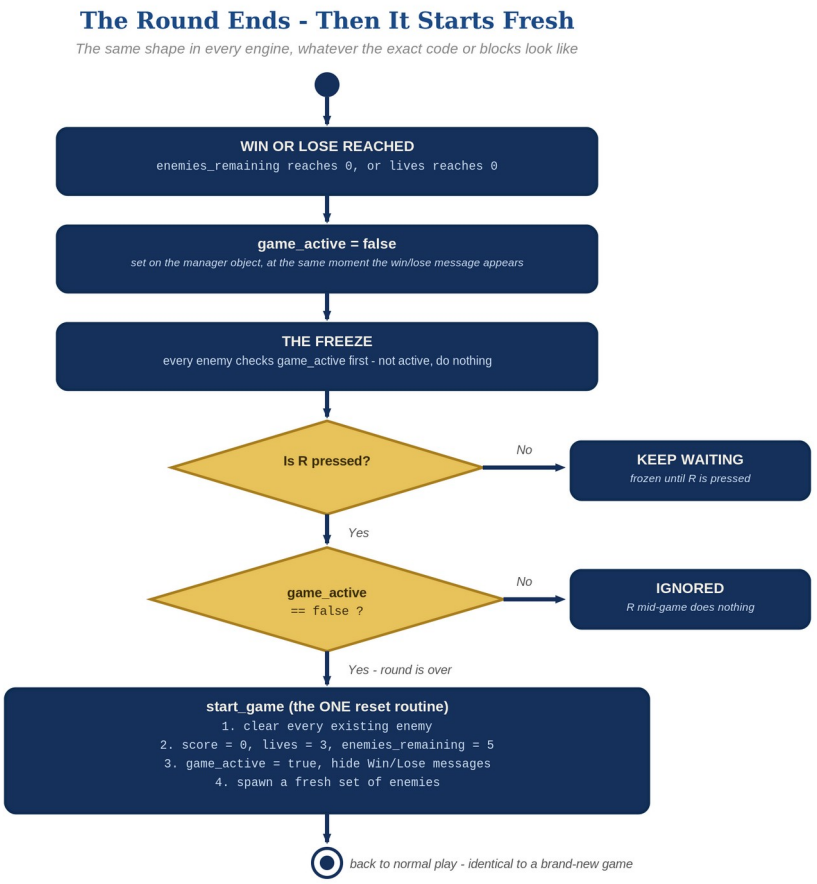
Tip: no new assets this week - this is a logic lesson, and it reuses your very first setup code from Lesson 5.

Setup

1. Open your [YourName]Game project from Lesson 9.
2. In GameManager's script, add var game_active := true near the top, with the rest of the shared game state.
3. Add the enemy scene (enemy.tscn) to a new Group named "enemies" (select the root node, open the Groups tab grouped with Inspector and Signals, type "enemies", click Add) - this lets you find and clear all enemies at once later.

Level 1 - Minimal (MVP)

When the round ends (win or lose), the game freezes - score and lives stop changing, and further enemy hits/escapes do nothing. Pressing R resets everything and starts a new round.



When the round ends, everything freezes - and R, pressed only while the round is over, runs the one reset routine. The same shape in every engine.

1. In the Main scene's script, move your spawn loop out of _ready() and into a new function spawn_enemies() (code below). Delete the old _ready() from the Main script - GameManager will now start the game.
2. In GameManager's script, add a new function start_game() that clears any existing enemies, resets score, lives, enemies_remaining, and game_active, hides WinText and LoseText, updates the HUD, and then asks the Main scene to spawn a fresh set of enemies (code below).
3. Change GameManager's _ready() to just call start_game.call_deferred(). Why the deferral: a child's _ready() runs BEFORE its parent has finished setting up, and a parent that is still busy refuses add_child() - so calling start_game() directly would silently spawn no enemies. call_deferred() waits one moment, until the whole scene has finished loading, and then runs it.
4. In enemy.tscn's _process(), add "if not get_node("../GameManager").game_active: return" as the very first line - this freezes movement once the round ends.
5. In on_enemy_destroyed() and on_enemy_escaped(), when the win or lose condition is reached, also set game_active = false and set the WinText/LoseText text to include the restart hint.

6. Add an _input(event) function to GameManager's script that checks for the R key and calls start_game() if the game isn't active (code below).
7. Run the scene, play to a win or loss, confirm movement freezes, then press R and confirm everything resets and play resumes.
8. Keep your sounds: if you added WinSound, LifeLostSound and GameOverSound in Lessons 6 and 8 (Level 3), they stay. The code below shows their play() lines as plain, unhighlighted lines. If you skipped that goal, leave those lines out - $WinSound and the others do not exist in your scene.

Highlighted lines are new this week; everything else is code you already have.

```
# GameManager's script (complete):
var enemies_remaining := 5
var lives := 3
var score := 0
var game_active := true

func _ready():
    start_game.call_deferred()

func start_game():
    for old_enemy in get_tree().get_nodes_in_group("enemies"):
        old_enemy.queue_free()
    enemies_remaining = 5
    lives = 3
    score = 0
    game_active = true
    get_node("../WinText").hide()
    get_node("../LoseText").hide()
    update_hud()
    get_parent().spawn_enemies()

func _input(event):
    if event is InputEventKey and event.pressed and event.keycode ==
KEY_R:
        if not game_active:
            start_game()

func update_hud():
    get_node("../ScoreText").text = "Score: " + str(score)
    get_node("../LivesText").text = "Lives: " + str(lives)

func on_enemy_destroyed():
    enemies_remaining -= 1
    score += 10
    update_hud()
    if enemies_remaining <= 0:
        game_active = false
        get_node("../WinText").text = "You Win! Press R to Restart"
        get_node("../WinText").show()
        $WinSound.play()  # your Lesson 6 Level 3 sound

func on_enemy_escaped():
    lives -= 1
    update_hud()
    if lives <= 0:
```

```
        game_active = false
        get_node("../LoseText").text = "Game Over - Press R to Restart"
        get_node("../LoseText").show()
        $GameOverSound.play()  # your Lesson 8 Level 3 sound
    else:
        $LifeLostSound.play()  # your Lesson 8 Level 3 sound

# Main scene's script (complete):
var enemy_scene := preload("res://enemy.tscn")

func spawn_enemies():
    for enemy_count in range(5):
        var new_enemy := enemy_scene.instantiate()
        new_enemy.position = Vector2(150 + enemy_count * 200, 50)
        add_child(new_enemy)
        new_enemy.destroyed.connect($GameManager.on_enemy_destroyed)
        new_enemy.escaped.connect($GameManager.on_enemy_escaped)

# enemy.tscn's script - add as the very first line of _process():
func _process(delta):
    if not get_node("../GameManager").game_active:
        return
    position.y += enemy_speed * delta
    if position.y > 700:
        escaped.emit()
        queue_free()
```

Before You Test: Audit Your Game State

Game state is everything your game remembers right now: score, lives, positions, costumes and colors, animations, sounds, and messages on screen. Until this lesson, your game started once and ended once, so nothing ever had to be put back. A restart means ALL of it must go back to how it was at the start - including anything extra you added that this handout doesn't mention.

1. Find EVERY _ready() in every script (Main, GameManager, Player, Enemy, and any extra node you added). _ready() runs only once, when a node first loads, so anything it sets up will NOT run again on restart. Move the setup that must repeat into start_game(), or call a reset function from start_game().
2. Look at everything that happens at game over: Tweens, AnimationPlayer animations, modulate or color changes, hidden or shown nodes, sounds, timers you started.
3. For each, add the opposite to start_game(): stop and reset the animation, set modulate back to white, show or hide the node, stop the sound, reset or stop the timer, and put the player back at its starting position.
4. Remember state lives on nodes other than GameManager too (the player's position, an Enemy's speed). If you changed it during play or at game over, reset it.

Rule of thumb: if it changed when the round ended (or at any point during play), something has to change it back when the round restarts.

Level 2 - Complete (Alpha/Beta)

A restart is a genuinely clean slate - no leftover enemies, no stale score/lives, and it works correctly no matter how many times in a row you play.

- Play to a win, confirm movement freezes, then press R and confirm score/lives/enemies_remaining are exactly back to 0/3/5 with no leftover enemy nodes.
- Repeat for a loss - same clean reset.
- Press R while the game is still active and confirm NOTHING happens - the "if not game_active" check in _input() is what prevents an accidental reset.

Level 3 - Refined (Ready to Publish) - pick any that interest you

- Fix a real bug! We laid the groundwork for this bug all the way back in Lesson 6 when we defined our win scenario. Then in Lesson 8 we defined our lose scenario. By thinking in terms of win and lose separately, rather than in terms of wave over and game over, we created a bug. Find it and resolve it in the manner you, the Designer, think best. (Hint: play a round where you destroy some enemies and let the others escape, and watch what happens to the round, and to your R key.)
- Visual: fade WinText/LoseText in instead of an instant show(), using a Tween.
- Structure: notice start_game() is now the single place that sets up a fresh game - called both from GameManager's _ready() and from its own _input() restart handling. No setup code is duplicated anywhere, and every piece of shared game state still lives on GameManager.

If Something's Wrong

- Restart does nothing when you press R: confirm the keycode comparison uses KEY_R (capital, an exact Godot constant) and that game_active is actually false at that point.
- Restart leaves old enemies on screen: confirm every spawned enemy was actually added to the "enemies" group - a node not in the group won't be found and cleared by get_nodes_in_group().
- Enemies still move after game-over: confirm the "if not ... game_active: return" line is the very FIRST line inside _process(), before any movement code runs.
- No enemies appear at start but no error shows in the game: check the Debugger > Errors tab for "Parent node is busy setting up children". It means start_game() was called directly from _ready(); change it to start_game.call_deferred().
- Error calling spawn_enemies(): confirm the function is defined in the Main scene's script (GameManager reaches it through get_parent()), and that GameManager is a direct child of the Main scene.

Want More?

Student Resource List > Godot section, or docs.godotengine.org